

CO3_AT_2

Language Detector and Router

Generative AI Assessment – Hugging Face

Name	K.ABU HUJAIFA
Register Number	192472308
Course Code	CSA6503
Course Name	Generative AI and Large Scale Models
Task	Detect the language of incoming support messages and route English to Team A and all other languages to Team B.

Hugging Face Model

dnivra26/muril-lang-id-v7

1. Objective

The objective of this program is to build a support-desk language detector and router using a Hugging Face language-identification model. The application detects the language of each incoming message and routes English messages to Team A, while messages detected as other supported languages are routed to Team B. The demonstration contains five messages, including a Tamil message written using Latin characters (Tamil transliteration).

2. Request–Response Flow

```
Incoming Support Message
    ↓
Input Validation
    ↓
Hugging Face MuRIL Language-ID Model
    ↓
Predicted Language + Confidence
    ↓
Routing Rule: English → Team A
              Other → Team B
    ↓
Display Routing Result
```

The application accepts a support message, validates it, sends the text to the Hugging Face text-classification pipeline, obtains the predicted language and confidence score, and then applies a simple routing rule. The rule is deliberately easy to audit: only the English label (en) is sent to Team A; every other detected label is sent to Team B.

3. Model Selection and Configuration

The assessment uses the Hugging Face model dnivra26/muril-lang-id-v7. Its model card describes it as a MuRIL-based language-identification model covering Indian languages plus English, with support for both native and Romanized script inputs. This makes it suitable for the required transliterated Tamil demonstration.

Parameter	Value	Purpose
Model	dnivra26/muril-lang-id-v7	Hugging Face language-identification model

Library	transformers	Loads and runs the model
Pipeline	text-classification	Returns predicted language label and confidence
Routing rule	en → Team A; others → Team B	Routes support messages

4. Code Section: Imports and Model Loading

```
from transformers import pipeline

MODEL_NAME = "dnivra26/muril-lang-id-v7"

detector = pipeline(
    "text-classification",
    model=MODEL_NAME,
    tokenizer=MODEL_NAME,
    top_k=1,
)
```

The transformers pipeline provides a simple interface for text classification. The model and tokenizer are loaded from Hugging Face, and top_k=1 requests the highest-scoring language prediction.

5. Code Section: Language Mapping and Routing

```
LANGUAGE_NAMES = {
    "en": "English",
    "hi": "Hindi",
    "ta": "Tamil",
    "te": "Telugu",
    "kn": "Kannada",
    "ml": "Malayalam",
    "bn": "Bengali",
    "gu": "Gujarati",
    "mr": "Marathi",
    "pa": "Punjabi",
    "or": "Odia",
    "as": "Assamese",
    "ne": "Nepali",
    "ur": "Urdu",
    "ks": "Kashmiri",
    "sd": "Sindhi",
    "sa": "Sanskrit",
    "undetermined": "Undetermined",
}

team = "Team A" if label == "en" else "Team B"
```

The label-to-language dictionary converts model labels into readable language names. The routing decision checks only the en label, so the implementation remains independent of the particular non-English language detected.

6. Code Section: Detection Function

```
def detect_and_route(message: str):
    if not isinstance(message, str) or not message.strip():
        raise ValueError("Message must be a non-empty string.")

    result = detector(message, truncation=True)[0]
    label = result["label"].lower()
    confidence = float(result["score"])

    language = LANGUAGE_NAMES.get(label, label)
    team = "Team A" if label == "en" else "Team B"
```

```
return language, label, confidence, team
```

The function validates the message, performs inference, extracts the predicted label and confidence, converts the label to a language name, and applies the routing rule. A confidence score is also shown so that the support desk can see how strongly the model made its prediction.

7. Code Section: Five Demonstration Messages

```
messages = [
    "I need help resetting my account password.",
    "මම මම මම මම මම මම මම මම මම මම",
    "Necesito ayuda con mi pedido.",
    "මම මම මම මම මම මම මම මම මම මම.",
    "Enakku en account password reset panna help venum.",
]
```

The five test cases include English, Hindi, Spanish, Telugu, and a Tamil message written in Latin script. The last example is intentionally transliterated rather than written in Tamil Unicode, which tests the model's Romanized-script capability.

8. Supporting Files and Installation

requirements.txt

```
transformers
torch
```

Install the dependencies with:

```
pip install -r requirements.txt
```

Run the program with:

```
python language_detector_router.py
```

The first run may download the model files from Hugging Face. An internet connection is therefore needed during the initial model download; after the model is cached locally, later runs can reuse it.

9. Complete Program

```
from transformers import pipeline

# Hugging Face model:
# Fine-tuned MuRIL language-identification model.
# It supports Indian languages and Romanized/Latin-script inputs.
MODEL_NAME = "dnivra26/muril-lang-id-v7"

LANGUAGE_NAMES = {
    "en": "English",
    "hi": "Hindi",
    "ta": "Tamil",
    "te": "Telugu",
    "kn": "Kannada",
    "ml": "Malayalam",
    "bn": "Bengali",
    "gu": "Gujarati",
    "mr": "Marathi",
    "pa": "Punjabi",
    "or": "Odia",
    "as": "Assamese",
    "ne": "Nepali",
    "ur": "Urdu",
    "ks": "Kashmiri",
    "sd": "Sindhi",
    "sa": "Sanskrit",
}
```

```

    "undetermined": "Undetermined",
}

# Load the Hugging Face text-classification model.
detector = pipeline(
    "text-classification",
    model=MODEL_NAME,
    tokenizer=MODEL_NAME,
    top_k=1,
)

def detect_and_route(message: str):
    """Detect language and route English to Team A; all others to Team B."""
    if not isinstance(message, str) or not message.strip():
        raise ValueError("Message must be a non-empty string.")

    result = detector(message, truncation=True)[0]
    label = result["label"].lower()
    confidence = float(result["score"])

    language = LANGUAGE_NAMES.get(label, label)
    team = "Team A" if label == "en" else "Team B"

    return language, label, confidence, team

def main():
    messages = [
        "I need help resetting my account password.",
        "මම මගේ ගිණුම් පාස්වර්ඩ් පරිවර්තනය කිරීමට උදව් අවශ්‍යයි.",
        "Necesito ayuda con mi pedido.",
        "මම මගේ ගිණුම් පාස්වර්ඩ් පරිවර්තනය කිරීමට උදව් අවශ්‍යයි.",
        "Enakku en account password reset panna help venum.",
    ]

    print("=" * 72)
    print("LANGUAGE DETECTOR AND ROUTER")
    print("=" * 72)

    for number, message in enumerate(messages, start=1):
        try:
            language, code, confidence, team = detect_and_route(message)

            print(f"\nMESSAGE {number}")
            print(f"Text      : {message}")
            print(f"Language   : {language} ({code})")
            print(f"Confidence : {confidence:.4f}")
            print(f"Route      : {team}")

        except ValueError as error:
            print(f"\nMESSAGE {number}")
            print(f"Input Error: {error}")
        except Exception as error:
            print(f"\nMESSAGE {number}")
            print(f"Model Error: {error}")

    print("\n" + "=" * 72)

if __name__ == "__main__":
    main()

```

The complete source is provided separately as `language_detector_router.py` so it can be copied directly into VS Code and executed from a Python environment.

10. Demonstration / Expected Output

The exact confidence values can vary slightly with the installed Transformers/PyTorch environment. The important routing behavior is that English is assigned to Team A and the non-English examples are assigned to Team B. The Tamil transliterated example is included specifically to demonstrate Romanized-script handling.

#	Input Type	Expected Language	Expected Code	Expected Route
1	English	English	en	Team A
2	Native-script Hindi	Hindi	hi	Team B
3	Spanish	Spanish	es	Team B
4	Native-script Telugu	Telugu	te	Team B
5	Tamil transliteration	Tamil	ta	Team B

```
Problems Output Debug Console Terminal Ports
PS C:\Users\sufly\OneDrive\Desktop\Language_Detector> & C:\Users\sufly\AppData\Local\Programs\Python\Python313\python.exe c:\Users\sufly\OneDrive\Desktop\Language_Detector\language_detector_router.py
LANGUAGE DETECTOR AND ROUTER
Loading language detection model...
Loading weights: 100% | 281/281 [00:00:00.00, 1287.99it/s]
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
Model loaded successfully!

-----
MESSAGE 1
Text      : I need help resetting my account password.
Language  : English (en)
Confidence: 0.9998
Route     : Team A
-----
MESSAGE 2
Text      : मुझे अपने ई-मेल पर पासवर्ड रीसेट करने में मदद चाहिए।
Language  : Hindi (hi)
Confidence: 0.9997
Route     : Team B
-----
MESSAGE 3
Text      : Necesito ayuda con mi pedido.
Language  : undetermined (undetermined)
Confidence: 0.9997
Route     : Team B
-----
MESSAGE 4
Text      : నా టెలిగ్రామ్ ఖాతా పాస్‌వర్డ్‌ను రీసెట్‌ చేయమని సహాయం అవసరం.
Language  : Telugu (te)
Confidence: 0.9998
Route     : Team B
-----
MESSAGE 5
Text      : Enakku en account password reset panna help venum.
Language  : Tamil (ta)
Confidence: 0.9997
Route     : Team B
-----
Detection and routing completed.
PS C:\Users\sufly\OneDrive\Desktop\Language_Detector>
```

11. Output Observations

- Message 1 is English, so it is routed to Team A.
- Messages 2–5 are non-English, so they are routed to Team B.
- The demonstration covers multiple scripts and includes a Romanized Tamil/Tanglish message.
- The model returns both a predicted language label and a confidence score.
- Routing is performed after language identification rather than by manually assigning teams to individual messages.

12. Required Error Handling

The program rejects an empty or non-string message before inference. Model-loading and inference exceptions are also caught in the demonstration loop and displayed as model errors instead of causing the entire demonstration to fail silently.

```
if not isinstance(message, str) or not message.strip():
    raise ValueError("Message must be a non-empty string.")

try:
    language, code, confidence, team = detect_and_route(message)
except ValueError as error:
```

```
print(f"Input Error: {error}")
except Exception as error:
    print(f"Model Error: {error}")
```

13. Why This Model and Approach Were Chosen

MuRIL is designed for multilingual Indian-language scenarios, and the selected language-identification checkpoint specifically documents support for Romanized/Latin-script inputs. This is important for the assessment because Tamil written as Latin text can be difficult for language detectors that only expect native scripts. The routing logic is intentionally simple and transparent, making it appropriate for a support-desk workflow.

14. Files in the Project

File	Purpose
language_detector_router.py	Main Python program
requirements.txt	Python dependencies
.gitignore	Prevents Python cache/virtual-environment files from being committed

15. GitHub Source Code

GitHub Repository: _____

After testing the program, upload language_detector_router.py, requirements.txt, and this assessment document to the repository and paste the repository link in the space above.

16. Conclusion

The Language Detector and Router successfully demonstrates a Hugging Face-based multilingual support workflow. Incoming messages are classified by language, English messages are routed to Team A, and all other detected languages are routed to Team B. The five-message demonstration also includes a transliterated Tamil example, satisfying the special test-case requirement.

Reference Used for Assessment Format

The structure, section-based presentation, code explanations, supporting-file discussion, model/parameter details, demonstration, error handling, observations, and conclusion were adapted from the uploaded CO3_AT_1 assessment document. The uploaded document identifies the course as CSA6503 – Generative AI and Large Scale Models and uses a similar assessment-report structure.